Visual Explainer: Angular Manifold Routing

This document explains the Angular Manifold Routing architecture in an intuitive way. It is intended for engineers, technical managers, and curious readers who want to understand the core mechanism without diving deeply into mathematical formalism.

1. The Problem With Traditional AI Compute

Most modern neural networks perform computation uniformly. When an input enters the model, nearly the entire network activates. This means large amounts of compute are spent regardless of whether all parts of the model are necessary for a particular input.

For very large models this becomes extremely expensive. Systems with billions or trillions of parameters require massive GPU clusters and large energy consumption.

2. The Idea Behind Sparse Routing

Sparse architectures attempt to solve this problem by activating only the portions of the model that are relevant for a given input. Instead of running the entire model every time, the system routes inputs to specific computational components.

Many previous approaches rely on learned gating networks to decide which components should activate. Angular Manifold Routing instead relies on geometry.

3. Step One: Embedding Inputs

Every token (such as a word or symbol) is converted into a numerical vector called an embedding. These embeddings live in a high■dimensional space.

Similar tokens often occupy nearby regions of this space because they represent related concepts.

4. Step Two: Normalization

The embeddings are normalized so that their length becomes 1. After normalization, every embedding lies on the surface of a hypersphere.

This means the only remaining information is the *direction* of the vector. Magnitude no longer matters.

5. Step Three: Partitioning the Sphere

The hypersphere is divided into angular sectors. Each sector represents a routing bucket. Tokens are assigned to the sector closest to their direction.

If we create K sectors, we might expect tokens to distribute evenly across all K buckets. However, experiments show something very different.

6. The Surprising Discovery

Tokens do not spread evenly across the routing sectors. Instead they cluster heavily. The number of sectors that actually receive tokens grows much more slowly than expected.

Observed scaling law:

$$E(K) \approx c \cdot K^{0.572}$$

K = total available routing sectors E(K) = sectors that actually become active

7. What This Means In Practice

Available Sectors (K)	Active Sectors Observed
100	~40
1,000	~150
5,000	~350

Even though thousands of routing sectors may exist, only a fraction activate. This produces natural computational sparsity.

8. Hardware Consequences

Because fewer buckets activate, the model performs less computation overall. This produces several benefits:

• lower routing cost • better memory locality • fewer cache misses • reduced hardware utilization

9. Why Geometry Creates This Effect

Embeddings that represent related information tend to point in similar directions. When routing uses angular partitions, these embeddings naturally fall into the same sectors.

This means the routing sparsity is not artificially imposed. It emerges as a property of the geometry of the embedding space itself.

10. Architecture Summary

The Angular Manifold Router pipeline looks like this:

Step	Description
Embed	Convert tokens into vectors
Normalize	Place vectors on a hypersphere
Partition	Divide sphere into angular sectors
Route	Assign tokens to nearest sector
Compute	Run experts only on active sectors
Aggregate	Combine outputs into final result

The key advantage is that the number of active sectors grows sublinearly with capacity. This allows the architecture to scale more efficiently than traditional routing systems.